Agent Reliability Engineering

从方法论到开源平台的落地方案

agentctl — Make Your Agent Survive Production

Wei Nan Li

2026 年 5 月 6 日

Demo 做完之后，Agent 为什么活不下去

Demo 阶段

- model + prompt 即可出效果
- 跑一次给老板看 → Done
- 不需要考虑可靠性

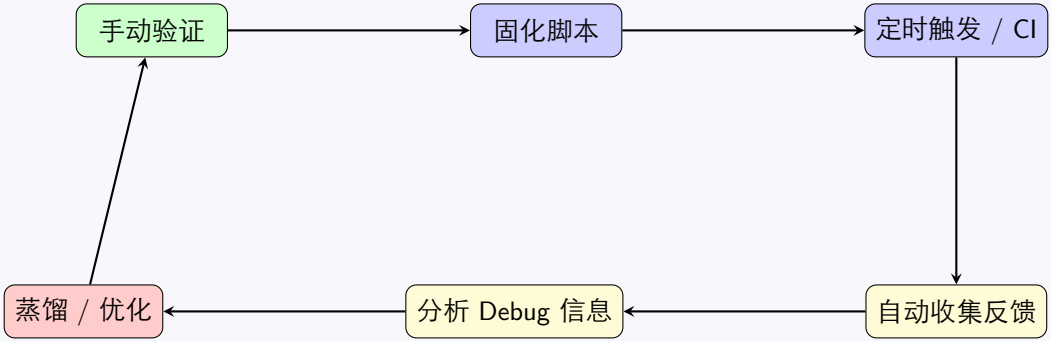
生产阶段

- 上下文爆炸，token 成本失控
- 同样的 prompt 昨天能用，今天崩了
- Agent 长期运行迷失方向
- compact 后丢失关键决策
- 没有人知道" 刚才发生了什么"

核心判断

Agent 工程化的最大瓶颈不是模型能力，而是缺乏一套让模型能够持续优化自身工作过程的工程体系。

自举 + 迭代：让 LLM 优化自身的工作流



不是一次性配置好就完事，而是构建一套**能持续自我改进的机制**

五层工程化实践（按成熟度排序）

5. 强类型驯服不确定性——Rust enum/Option 约束 LLM 输出结构

4. 埋点驱动迭代——三层 debug 信息 → 数据飞轮

3. Compact 卡点解法——混合记忆策略：压缩前提取 + 压缩后注入

2. tmux 作为 Agent 运行时——持久化 + 进程管理 + 脚本观测

1. 复杂流程固化——脚本 / API，不让 LLM 重复学习

每层的共性不是"让模型更强"，而是**把人的工程判断逐步沉淀为系统能力。**

差异化记忆点：tmux 作为 Agent 运行时

为什么用 tmux?

- **持久化上下文**：LLM 状态不因单次 API 调用结束而丢失
- **脚本可观测**：capture-pane 读取输出，不干扰推理
- **崩溃恢复**：tmux session 独立于进程
- **交互信道**：send-keys 注入指令，无需重新建连

行业中独一无二

没有人用 tmux 做 agent runtime，但它恰好解决了"agent 崩溃恢复、脚本观测、不打断上下文" 三个痛点。

示例

```
tmux send-keys -t claude "analyze logs" Enter
tmux capture-pane -t claude -p > latest_output.txt
```

竞争定位：Agent 可靠性工程

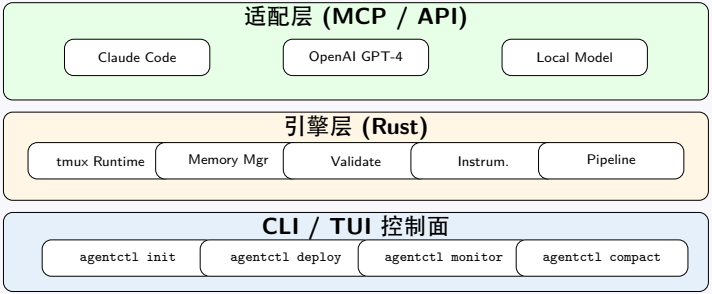
竞品对标矩阵

	LangSmith	CrewAI	Claude Code	Dify	你
可观测性/埋点	✓	✗	✗	✓	✓
Agent 持久化运行时	✗	✗	部分	✗	✓
Compact 策略	✗	✗	内置	✗	✓
自举方法论	✗	✗	✗	✗	✓
跨模型适配	✓	✓	✗	✓	✓
私有化部署	企业版	✗	✗	企业版	✓
强类型校验	✗	✗	✗	✗	✓
开源	✗	✓	✗	✓	✓

关键发现

Agent 可靠性工程这个品类目前**有需求但没有人系统性地做**。LangSmith 偏观测，Dify 偏应用搭建，中间地带是空白。

产品形态：三层架构



- 引擎层用 Rust 实现，单二进制分发，零运行时依赖（除 tmux）
- 适配层通过 MCP 协议对接不同模型供应商
- CLI 是所有操作的门面，不依赖 Web UI

Phase 1：开源核心 + 方法论品牌　　Month 1–3

做什么

- 开源 `agentctl` CLI（Rust，单二进制）
- 三个核心命令：`init` / `monitor` / `compact`
- 发布方法论白皮书（EN + CN）
- Hacker News / Reddit / 掘金推广

不要做什么

- 不要做可视化 UI
- 不要做 multi-agent 编排
- 不要承诺兼容所有模型

Phase 1 成功标准

GitHub $\geq$ 500 stars + 5 个团队在无帮助情况下自搭建并持续使用 $\geq$ 1 个月

Phase 2：SaaS 平台　　Month 4–9

- 搭建 `agentcloud.io`（或其他域名）
- 核心功能：Agent 托管 + 埋点 Dashboard + Compact 历史回溯
- **定价**：
  - Free tier：单 agent
  - Pro：$29/月，多 agent + 历史回溯
  - Enterprise：定制报价，私有部署
- 目标用户：3–10 人的 AI-native 小团队、独立开发者

关键指标（非收入）

付费转化率　+　月留存率

Phase 3：企业私有化部署 Month 10–18

- Docker Compose / Kubernetes Helm Chart 一键部署
- 企业版功能：SSO、审计日志、多租户、SLA
- **定价**：年合同 $10K–$50K，按 agent 数量 + 定制化
- 客户画像：**金融 / 医疗 / 制造业**有内部 LLM 工具团队的客户

第一个客户靠自己做

不要等 inbound。用现有公司关系找一个 Design Partner，免费或低价部署，把 case 做扎实。一个真实的客户案例比 1000 个 star 更有说服力。

机会 vs 风险

有利因素 ↑

1. **时机**：2026 年是 agent 从玩具到工具的关键年
2. **差异化**：tmux 运行时记忆点独特，易在技术社区传播
3. **门槛低**：Phase 1 只需 1 人 + 现有公司资源
4. **已有公司**：可复用现有法务/财务基础设施

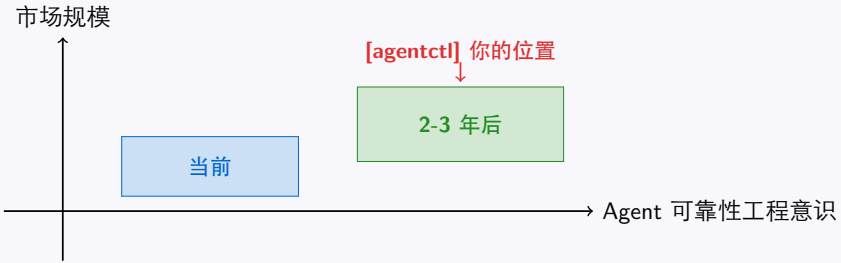
风险因素 ↓

1. **平台内建**：Anthropic/OpenAI 可能内建类似能力
2. **市场教育**：概念尚未形成共识
3. **变现路径长**：开源 → 企业版需 3–5 年
4. **双向竞争**：上有大平台，下有 DIY 脚本

核心风险应对

不要绑死在单一模型上。适配层从 Day 1 就支持多模型，Claude 内建同类能力时切换到差异化。

行业位置：定义者而非追随者



不是领导者，但有机会成为定义者。

前提：Phase 1 的内容输出要密集且高质量——博客、talk、开源文档三管齐下，让"Agent 可靠性工程" 这个概念和你的项目绑定。

务实建议：启动路线

1. **不要先搭平台再找用户。**
   Phase 1 的 CLI 开源就是验证——没人用 CLI，平台也不会有付费用户。
2. **用博客驱动增长。**
   每篇博文让读者"学到东西" + "知道你的工具能做什么"。
3. **第一个企业客户靠自己找。**
   用现有关系找一个 Design Partner，免费/低价部署，把 case 做扎实。
4. **衡量标准不是 star 数。**
   真正的指标：5–10 个团队在没你帮助的情况下自搭建并持续使用超一个月。

一句话

这个项目的天花板

取决于「**Agent 可靠性**」是否能成为行业公认的问题类别。

如果两年后每家公司都在讨论 Agent Observability 和 Reliability，
而你的项目是**第一个认真做这件事的**——
那你的位置就对了。

Thank You

github.com/liweinan

weinan.io